Supervised Learning COMP0048 : Coursework 1

Jordan Sydney-Darling 20127163, Daniel Pulido 25164517

November 2025

1 Part I

1.1 Linear Regression

Q1

(a)

Polynomial basis models of degree k = 1 2 3 4 fitted to the data

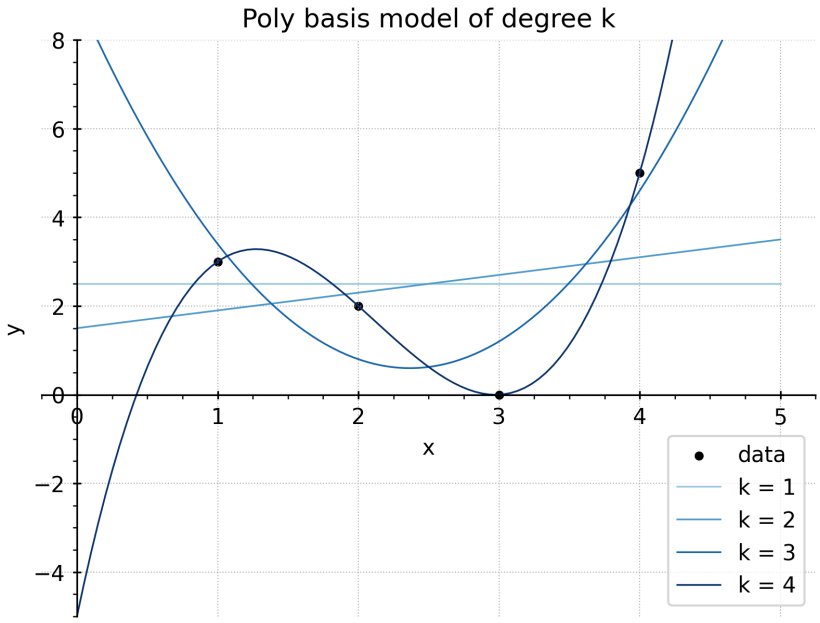


Figure 1: Fitted polynomial models of degree k on four data points. Higher degree models interpolate the data and achieve almost 0 training error.

(b)

The general polynomial regression model of degree k can be written as:

$$y(x) = w_0 + w_1 x + w_2 x^2 + \cdots + w_k x^k$$

Refer to Table 1 for the corresponding fitted weights.

(c)

MSE of the models

k	w_0	w_1	w_2	w_3	mse
1	2.5000				3.25
2	1.5000	0.4000			3.05
3	9.0000	-7.1000	1.5000		0.8
4	-5.0000	15.1667	-8.5000	1.3333	$6.98063e-27$

Table 1: Weights, w_i, and training error for polynomial models of degree k.

Q2

(a)

(i)

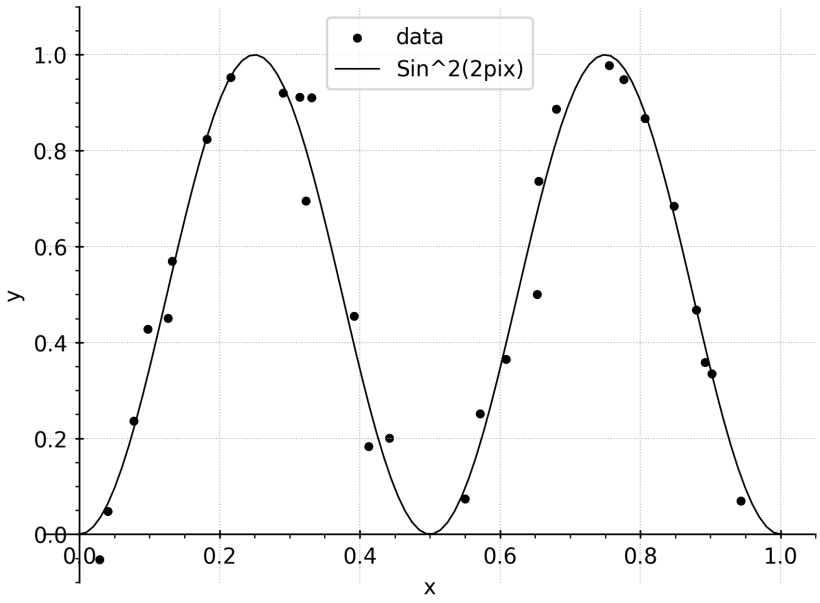


Figure 2: Generated samples from g(x)=$sin^2(2\pi x)$ with Gaussian noise, alongside the true underlying function.

(ii)

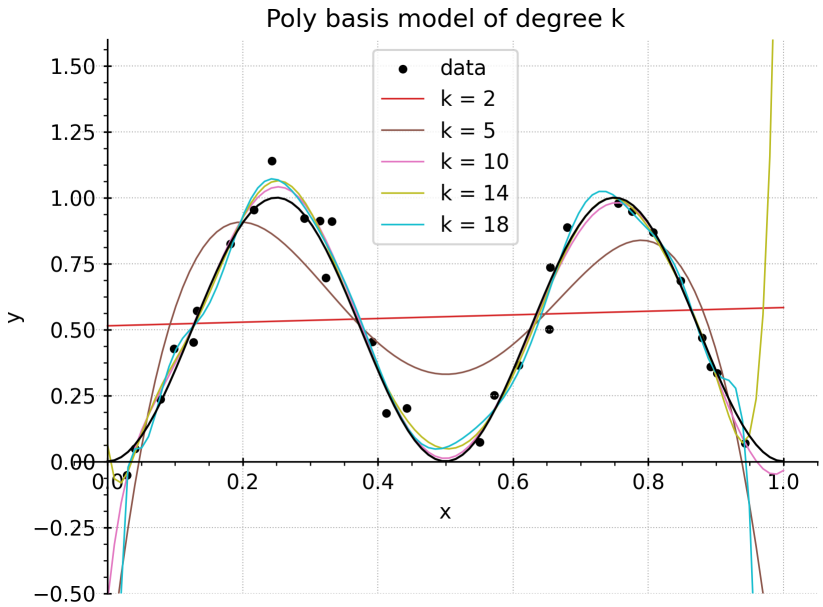


Figure 3: Polynomial models of increasing degree fitted to noisy samples. Underfitting occurs at low k, higher k captures noise and overfits.

(b)

Variation in the training error with the degree, k, of the polynomial basis functions

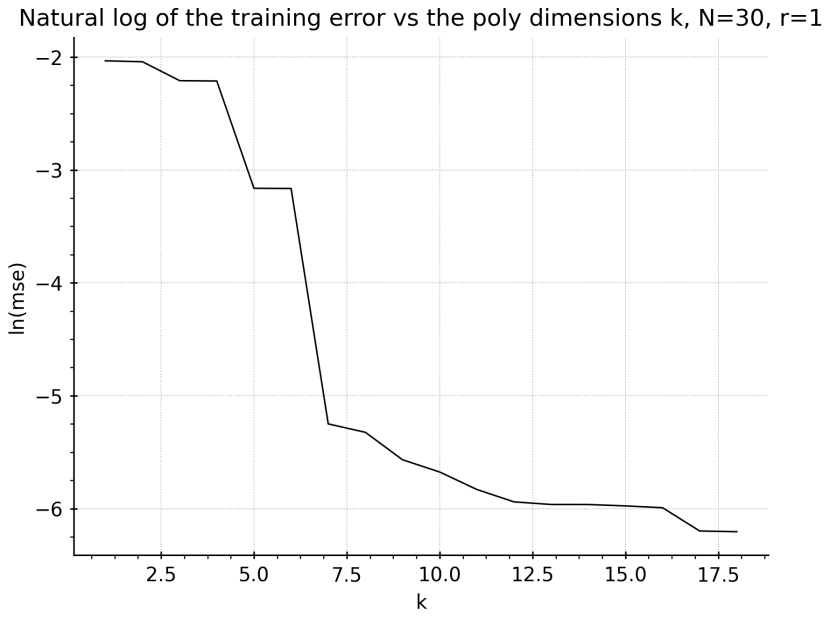


Figure 4: Training error decreases monotonically with increasing polynomial degree due to increasing model flexibility.

(c)

Variation in the test error with the degree, k, of the polynomial basis functions

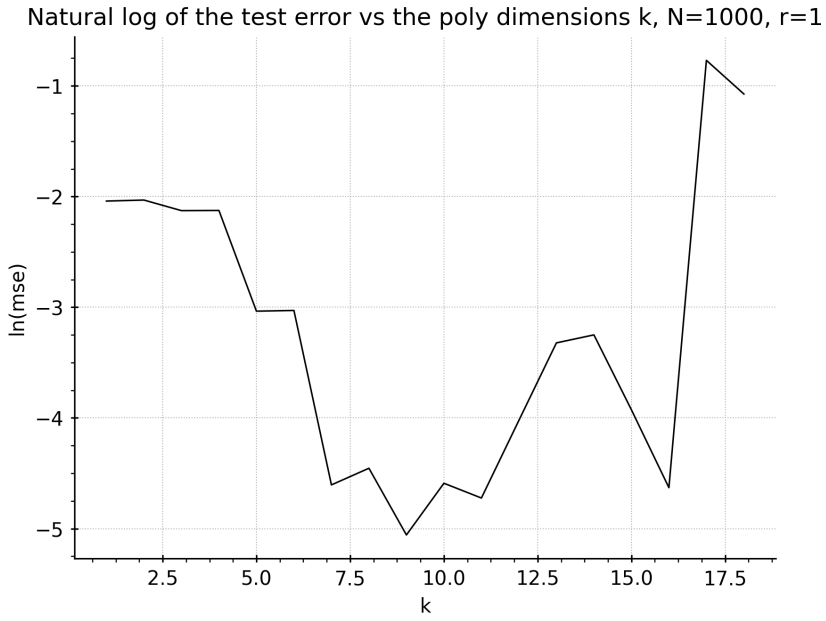


Figure 5: Test MSE for a single run of polynomial regression, showing how increasing model degree first reduces error before overfitting increases it again.

(d)

Variation in the training and test error with the degree, k, of the polynomial basis functions, averaged over 100 runs

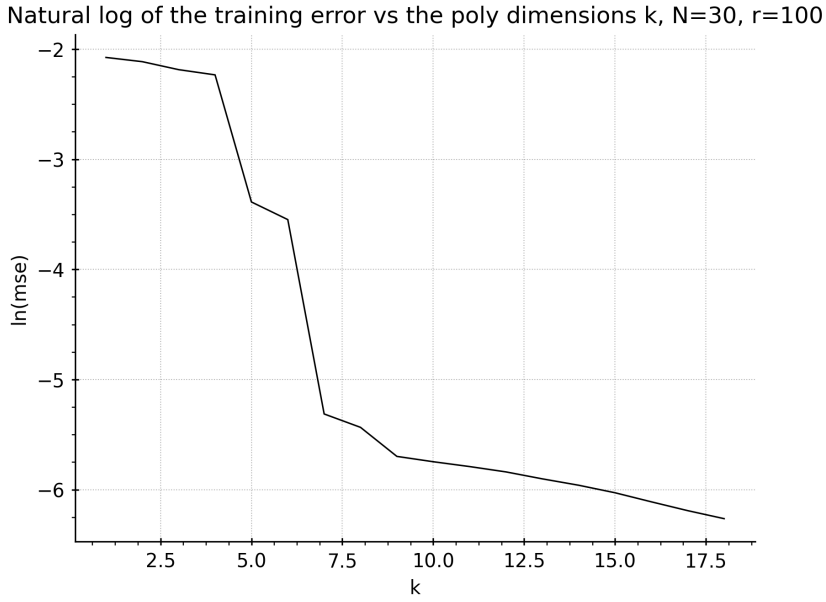


Figure 6: Average training MSE over 100 repetitions, illustrating the monotonic decrease in error as model complexity increases.

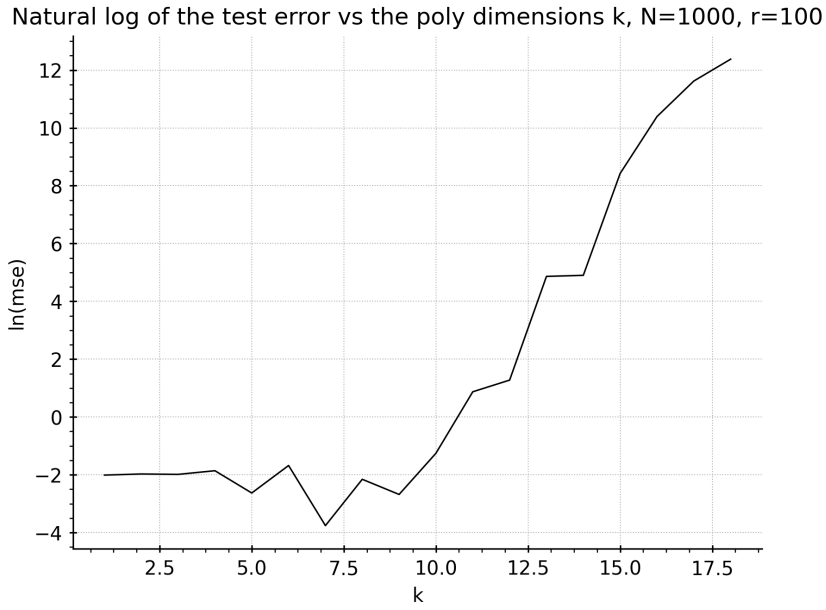


Figure 7: Average test MSE over 100 repetitions, showing a U-shaped curve where moderate polynomial degrees generalise best.

k	Train $N = 30, r = 1$	Test $N = 1000, r = 1$	Train $N = 30, r = 100$	Test $N = 1000, r = 100$
1	0.1308	0.1299	0.1255	0.1340
2	0.1297	0.1311	0.1208	0.1391
3	0.1097	0.1192	0.1123	0.1373
4	0.1094	0.1194	0.1071	0.1558
5	0.0423	0.0481	0.0338	0.0721
6	0.0422	0.0484	0.0288	0.1866
7	0.0052	0.0100	0.0049	0.0233
8	0.0049	0.0116	0.0044	0.1158
9	0.0038	0.0064	0.0034	0.0686
10	0.0034	0.0102	0.0032	0.2852
11	0.0029	0.0089	0.0031	2.3913
12	0.0026	0.0179	0.0029	3.5805
13	0.0026	0.0361	0.0027	129.4492
14	0.0026	0.0388	0.0026	134.2667
15	0.0025	0.0197	0.0024	4557.1736
16	0.0025	0.0098	0.0022	32590.5203
17	0.0020	0.4633	0.0020	111462.0919
18	0.0020	0.3418	0.0019	237303.6191

Table 2: Training and test error, MSE, for polynomial models of degree k, N data points, and r runs.

Q3 Sinusoidal Model

Repeat of Q2 (b-d) using the basis $\{\sin(\pi x), \sin(2\pi x), \sin(3\pi x), \ldots, \sin(k\pi x)\}$

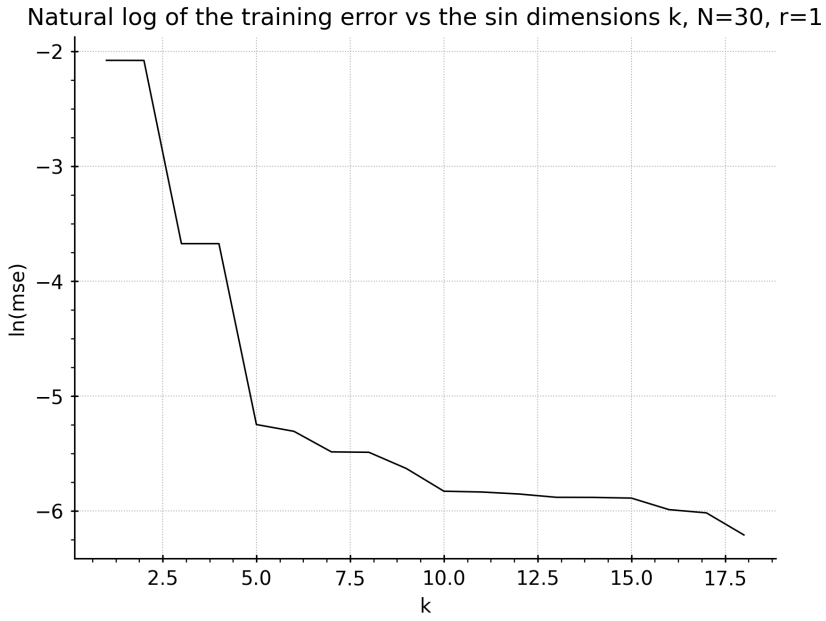


Figure 8: Training MSE for a single run using sinusoidal basis, showing decreasing error as k increases

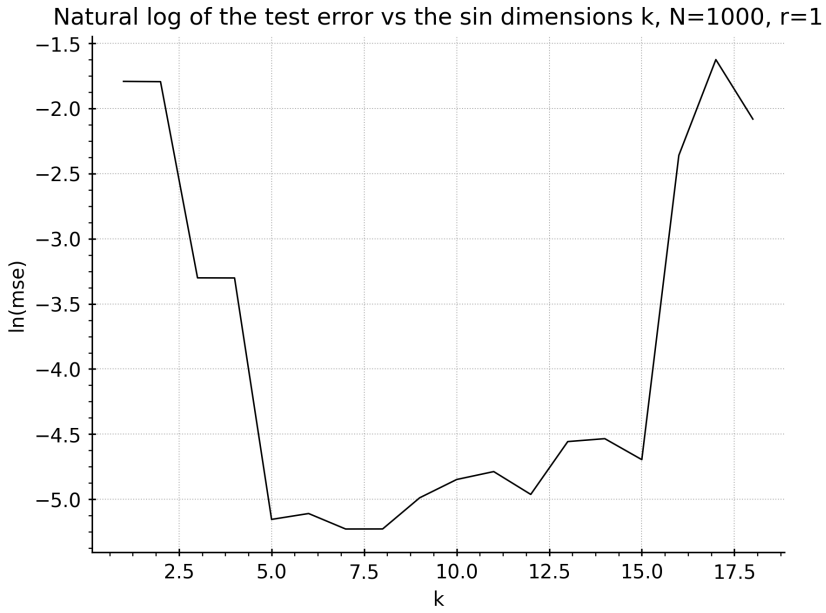


Figure 9: Average test MSE over a single run using sinusoidal basis, with smallest error at medium k and overfitting for large k

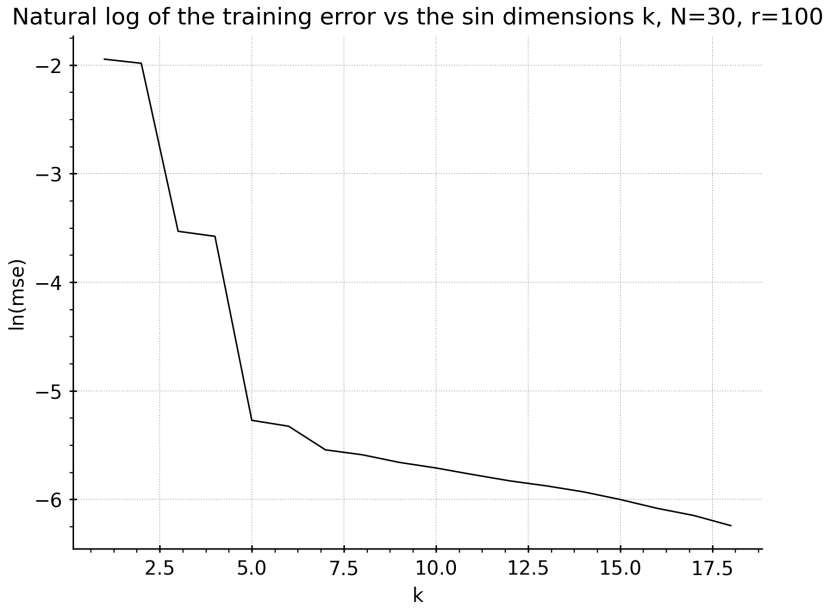


Figure 10: Average training MSE over 100 repetitions using sinusoidal basis, consistently decreasing with model complexity

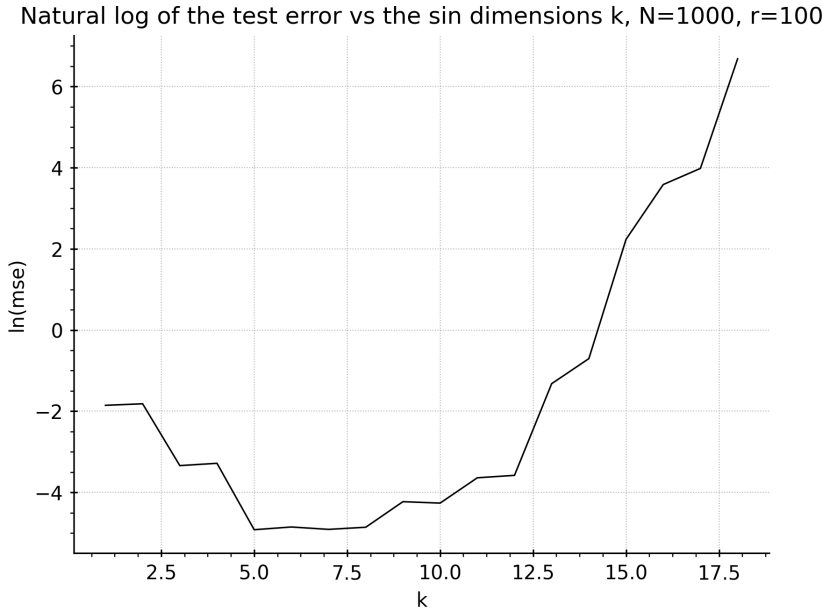


Figure 11: Average test MSE over 100 repetitions using sinusoidal basis, again showing best generalisation at moderate k

k	Train $N = 30$, $r = 1$	Test $N = 1000$, $r = 1$	Train $N = 30$, $r = 100$	Test $N = 1000$, $r = 100$
1	0.1250	0.1667	0.1428	0.1565
2	0.1249	0.1663	0.1375	0.1625
3	0.0253	0.0369	0.0293	0.0354
4	0.0253	0.0369	0.0279	0.0375
5	0.0053	0.0058	0.0051	0.0073
6	0.0050	0.0060	0.0049	0.0078
7	0.0041	0.0054	0.0039	0.0074
8	0.0041	0.0054	0.0037	0.0078
9	0.0036	0.0068	0.0035	0.0146
10	0.0029	0.0078	0.0033	0.0141
11	0.0029	0.0083	0.0031	0.0262
12	0.0029	0.0070	0.0029	0.0279
13	0.0028	0.0105	0.0028	0.2668
14	0.0028	0.0107	0.0027	0.4946
15	0.0028	0.0091	0.0025	9.3474
16	0.0025	0.0945	0.0023	36.0391
17	0.0024	0.1970	0.0021	53.6470
18	0.0020	0.1248	0.0019	798.3712

Table 3: Training and test error, MSE, for sinusoidal models of degree k, N data points, and r runs.

1.2 Filtered Boston housing and kernels

4

(a) Naive Regression, averaged over 20 runs

Train MSE $= 84.54296 \pm 5.52900$, Test MSE $= 84.47044 \pm 11.03504$, w_0 : 22.05312

(b)

The constant function of ones predicts the mean y value of the training data and ignores all the input features.

(c) Linear Regression with single attributes, averaged over 20 runs

Attribute	Train MSE	Test MSE	w_0	w_1
(0) CRIM	71.24928 ± 5.02151	73.91214 ± 10.67044	23.5254	-0.3956
(1) ZN	74.19339 ± 4.45628	72.38310 ± 8.99867	20.6052	0.1333
(2) INDUS	64.91554 ± 4.45945	64.55971 ± 9.08022	29.2453	-0.6327
(3) CHAS	82.01689 ± 5.25539	82.19306 ± 10.89841	21.6738	6.3912
(4) NOX	69.47586 ± 4.56053	68.48282 ± 9.20977	40.3537	-32.8778
(5) RM	43.35706 ± 3.09579	44.46112 ± 6.05924	-33.1037	8.8609
(6) AGE	72.68517 ± 4.93838	72.25894 ± 9.92758	30.4520	-0.1211
(7) DIS	79.43880 ± 5.39861	78.97677 ± 10.81822	17.8631	1.1124
(8) RAD	71.92846 ± 5.01080	73.07948 ± 10.10695	25.6182	-0.3816
(9) TAX	65.58703 ± 4.60109	66.96810 ± 9.25111	32.2209	-0.0248
(10) PTRATIO	62.35007 ± 4.09324	63.83268 ± 8.29972	60.4908	-2.0800
(11) LSTAT	38.69801 ± 2.39446	38.50403 ± 4.75156	34.1912	-0.9246

Table 4: Average training and test MSE (± one standard deviation) for linear regression using a single attribute plus bias, evaluated over 20 random train/test splits. Attribute numbers correspond to the Boston Housing column indices.

(d) Linear Regression with all attributes, averaged over 20 runs

Train MSE $= 22.27992 \pm 1.69230$, Test MSE $= 23.97525 \pm 3.67887$.

Weight	Value
w_0 (bias)	42.4269
w_1 (CRIM)	-0.1211
w_2 (ZN)	0.0350
w_3 (INDUS)	-0.0249
w_4 (CHAS)	2.1835
w_5 (NOX)	-17.2510
w_6 (RM)	3.6054
w_7 (AGE)	0.0046
w_8 (DIS)	-1.4812
w_9 (RAD)	0.2679
w_{10} (TAX)	-0.0101
w_{11} (PTRATIO)	-1.0167
w_{12} (LSTAT)	-0.5551

Table 5: Weights learned by the full linear regression model using all attributes.

1.3 Kernelised ridge regression

Q5

(a)

σ	$2^{12.5}$
γ	$2^{-39.0}$
Train MSE	11.55487
Test MSE	12.48412

(b)

3D plot of the cross-validation MSE as a function of σ and γ

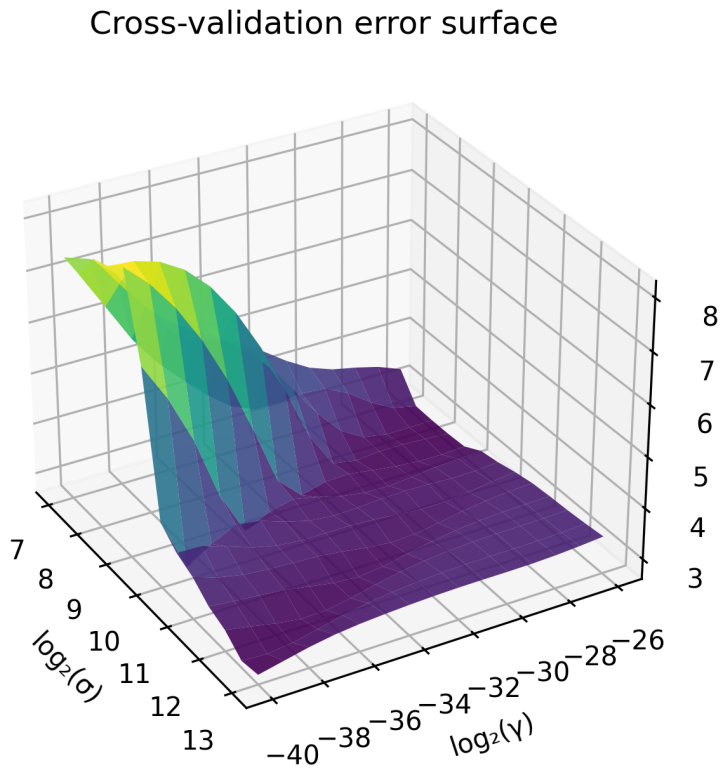


Figure 12: Cross-validation surface of $\log_2(\sigma)$ vs $\log_2(\gamma)$ showing test MSE.

(c)

Using the best σ and γ from Q5a : fig 12, the MSEs are as follows

Train MSE	11.55487
Test MSE	12.48412

(d)

Table of the training and test MSEs for all the different models, averaged and tested over 20 of the same splits

Method	Training MSE	Test MSE
Naive Regression	84.543 ± 5.529	84.47 ± 11.035
Linear Regression (attribute 0)	71.249 ± 5.0215	73.912 ± 10.67
Linear Regression (attribute 1)	74.193 ± 4.4563	72.383 ± 8.9987
Linear Regression (attribute 2)	64.916 ± 4.4594	64.56 ± 9.0802
Linear Regression (attribute 3)	82.017 ± 5.2554	82.193 ± 10.898
Linear Regression (attribute 4)	69.476 ± 4.5605	68.483 ± 9.2098
Linear Regression (attribute 5)	43.357 ± 3.0958	44.461 ± 6.0592
Linear Regression (attribute 6)	72.685 ± 4.9384	72.259 ± 9.9276
Linear Regression (attribute 7)	79.439 ± 5.3986	78.977 ± 10.818
Linear Regression (attribute 8)	71.928 ± 5.0108	73.079 ± 10.107
Linear Regression (attribute 9)	65.587 ± 4.6011	66.968 ± 9.2511
Linear Regression (attribute 10)	62.35 ± 4.0932	63.833 ± 8.2997
Linear Regression (attribute 11)	38.698 ± 2.3945	38.504 ± 4.7516
Linear Regression (all attributes)	22.28 ± 1.6923	23.975 ± 3.6789
Kernel Ridge Regression	8.34 ± 1.553	12.053 ± 1.9799

Table 6: Comparison of model performance across linear and kernel methods. Kernel ridge regression achieves the lowest average test MSE.

Kernel ridge regression achieved the lowest average test and train MSE.

2 Part II

2.1 *k*-Nearest Neighbors

Q6

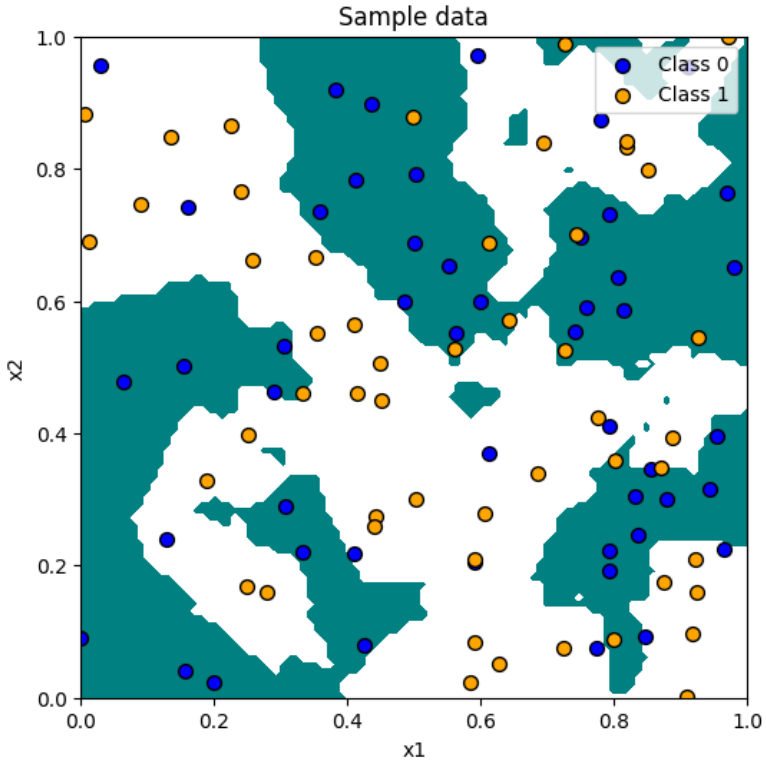


Figure 13: Decision regions of a 3-NN classifier on $[0, 1]^2$ with randomly sampled training data.

Q7

(a)

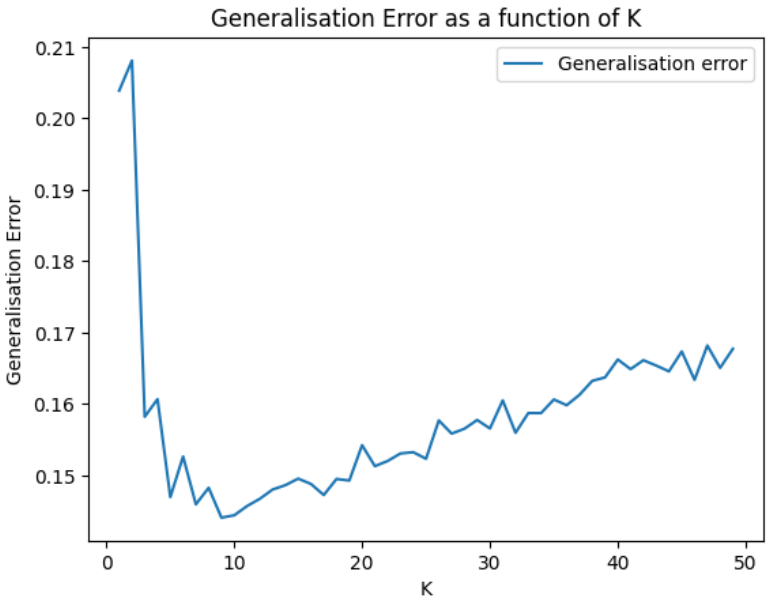


Figure 14: Estimated generalisation error of k-NN as a function of k using Protocol A.

(b)

It can be observed how the generalisation error decreases sharply as K increases, until K=9 is reached. From that point onwards, the generalisation error begins to increase linearly with K.

For low values of K, as the prediction depends on just a few points, this prediction can be affected by noisy points that may give a wrong prediction. As K increases, the effect of the noise is reduced, as more points are taken into account, so those nearest neighbors will have a higher chance of being representative of the general distribution of the nearby points. However, for large values of K the locality of the algorithm is lost, as points which are far away from the test point, which do not give any information about the actual point to be studied, are considered for the prediction, therefore decreasing its precision.

Q8

(a)

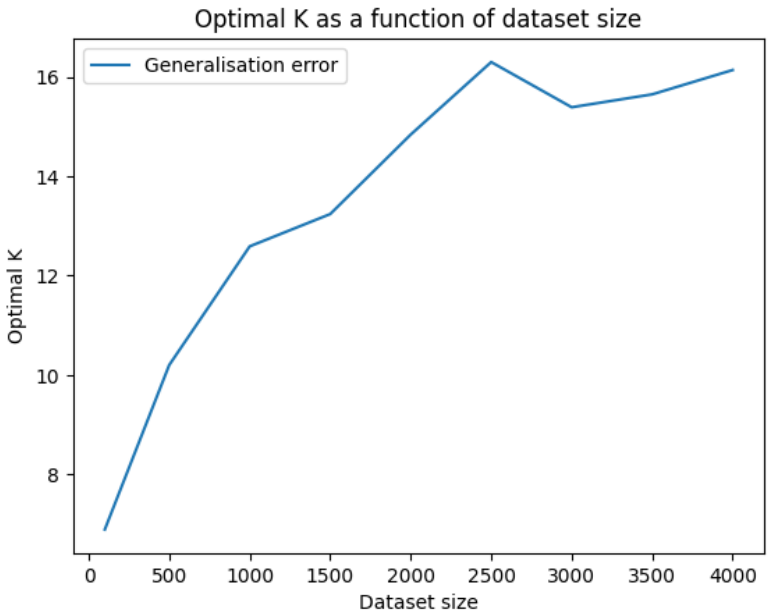


Figure 15: Optimal value of k as a function of training set size m under Protocol B.

(b)

The optimal value for K increases as the train dataset size gets bigger, until it reaches a maximum of K=16 at 2500 points. This increase is expected, as a bigger dataset implies a higher density of points, so more points will be needed cover the same area. However, the optimal value for K stabilises after a certain dataset size, as the amount of neighbors reaches a point in which it is enough to get a representative sample of the population surrounding the datapoint we want to classify, and no more information is needed to provide a good prediction.

3 Part III

3.1 Questions

Q9

(a)

$$K_c(x,z) = c + \sum_{i=1}^{n} x_i z_i, \quad x,z \in \mathbb{R}^n$$

Prove that $K_c(x,z)$ is positive semidefinite (PSD).

$$\text{PSD} \implies a^\top K a \geq 0 \quad \forall a \in \mathbb{R}^n$$

$$a^\top K a = \sum_{i,j=1}^{n} a_i a_j K(i,j)$$

$$= \sum_{i,j=1}^{n} a_i a_j \langle \phi(x_i), \phi(x_j) \rangle$$

$$= \left\langle \sum_{i=1}^{n} a_i \phi(x_i), \sum_{j=1}^{n} a_j \phi(x_j) \right\rangle$$

$$= \left\| \sum_{i=1}^{n} a_i \phi(x_i) \right\|^2 \geq 0$$

$\implies$ If we can show that $K_c(x,z)$ is an inner product of itself, it follows that

$$a^\top K a \geq 0 \implies \text{PSD.}$$

$$\langle \phi(x), \phi(z) \rangle = c + \sum_{i=1}^{n} x_i z_i$$

$$\sum_{i=1}^{n} x_i z_i = x^\top z = \langle x, z \rangle$$

$$\implies \langle \phi(x), \phi(z) \rangle = c + x^\top z = f(x) f(z) + x^\top z$$

i.e

$$\implies \phi(x) = \begin{bmatrix} x \\ f(x) \end{bmatrix}$$

$$c = f(x) f(z) \quad \forall x, z$$

$\implies$ function $f(\cdot)$ must be constant, $\implies c = f(\cdot)^2 \implies f(\cdot) = \sqrt{c}$

$$\phi(x) = \begin{bmatrix} x \\ \sqrt{c} \end{bmatrix}$$

$$\langle \phi(x), \phi(z) \rangle = \begin{bmatrix} x^\top & \sqrt{c} \end{bmatrix} \begin{bmatrix} z \\ \sqrt{c} \end{bmatrix} = x^\top z + (\sqrt{c})^2 = c + \sum_{i=1}^{n} x_i z_i = K_c(x,z)$$

If $c = 0$, then $K_c(x,z) = \sum_{i=1}^{n} x_i z_i = \langle x, z \rangle$ (linear kernel). Therefore, K_c is a positive semidefinite kernel.

If $c > 0$, then $K_c(x,z) = c + \sum_{i=1}^{n} x_i z_i = \langle \phi(x), \phi(z) \rangle$ with $\phi(x) = (\sqrt{c}, x_1, \ldots, x_n)$. Therefore, K_c is a positive semidefinite kernel.

If $c < 0$, the previous argument cannot be used, as $\sqrt{c} \notin \mathbb{R}$ when $c < 0$. In this case, it can be seen

that, when taking $x = 0$, $K_c(x,x) = K_c(0,0) = c < 0$. Therefore, K_c is not a positive semidefinite kernel when $c < 0$.

(b)

When $c = 0$, the corresponding linear regression is purely linear and forced through the origin, as there is no actual feature map applied over the data, while the case $c = 1$ is equal to the standard linear regression with bias.

Therefore, the variable c just adds a bias to the linear regression. The value of c only causes the weight associated to the bias to be divided by $\sqrt{c}$ with respect to the standard biased model (the one with $\phi(x) = (1,x)$). This causes high weights in the bias to be less penalised as c increases in the regularised linear regression, allowing for higher y-intercept values for the linear predictor.

Q10

With the Gaussian kernel, we have the following classifier: $f_\beta(t) = \sum_{i=1}^{m} \alpha_i e^{-\beta\|x_i - t\|^2}$.

Intuitively, when $\beta \to \infty$, the term $e^{-\beta\|x_i-t\|^2}$ will tend to 0, except for the cases in which $\|x_i - t\|^2 \approx 0$. Therefore, only the points which are a very small distance away from the point t will be considered. For a big enough value of β, the nearest neighbor will have a much higher value for $e^{-\beta\|x_i-t\|^2}$ than the rest, enabling the trained linear classifier to simulate 1-NN.

Let's give a more rigorous proof. Let x_n be the nearest neighbor, so that $\|x_n - t\|^2 \leq \|x_i - t\|^2$ for all $i \neq n$.

With this notation, $\lim_{\beta\to\infty} \frac{K_\beta(x_i,t)}{K_\beta(x_n,t)} = \lim_{\beta\to\infty} \frac{e^{-\beta\|x_i-t\|^2}}{e^{-\beta\|x_n-t\|^2}} = \lim_{\beta\to\infty} e^{-\beta(\|x_i-t\|^2 - \|x_n-t\|^2)} = 0$ for $i \neq n$, as $\|x_i - t\| - \|x_n - t\| > 0$.

Therefore, $f_\beta(t) = \alpha_n K_\beta(x_n,t) + \sum_{i\neq n} \alpha_i K_\beta(x_i,t) = K_\beta(x_n,t)[\alpha_n + \sum_{i\neq n} \alpha_i \frac{K_\beta(x_i,t)}{K_\beta(x_n,t)}]$. Taking limits:

$$\lim_{\beta\to\infty} f_\beta(t) = \lim_{\beta\to\infty} K_\beta(x_n,t)[\alpha_n + \sum_{i\neq n} \alpha_i \frac{K_\beta(x_i,t)}{K_\beta(x_n,t)}] = \alpha_n \lim_{\beta\to\infty} K_\beta(x_n,t).$$

We have proved that $\lim_{\beta\to\infty} f_\beta(t)$ only depends on the nearest neighbor. Now we must prove that the value assigned to the point t is exactly the label of the nearest neighbor.

If we take a fixed, but very high value of β, $f_\beta(t) \approx \alpha_n K_\beta(x_n,t)$, as we have proved beforehand. Training the linear regressor is equivalent to minimizing the least-squares error. Taking into account that the nearest neighbor to a training point is itself, the following least-squares error formula is obtained: $\sum_{i=1}^{m}(y_i - f_\beta(x_i))^2 = \sum_{i=1}^{m}(y_i - \alpha_i K_\beta(x_i,x_i))^2 = \sum_{i=1}^{m}(y_i - \alpha_i)^2$, as $K_\beta(x_i,x_i) = 1$. Thus, the least-square error is minimized by taking $\alpha_i = y_i$ for all i, so $f_\beta(t) \approx \alpha_n K_\beta(x_n,t) \approx y_n$ as $K_\beta(x_n,t) \approx 1$ if x_n and t are close enough.

Q11

We considering the problem of generalised Whack-A-Mole on an n x n board. We shall denote the state of each hole as follows.

1 if a mole is present
0 if a mole is hidden

Every time you hit a hole you'll cause the hole and the immediate four adjacent holes to change state, ie if you hit a hole in position (i, j), $i, j \in n$, all the holes $(i-1, j), (i+1, j), (i, j-1), (i, j+1), (i, j)$ will change state; 1s become 0s and vice versa.
Note that in the edge cases where an adjacent hole does not exist as it is outside the n x n board, we can just ignore that point.

We are tasked with designing an algorithm which, given an intial board configuration, finds a sequence of holes that can be hit in order to empty the board if such a sequence exists. Additionally the algorithm must be polynomial in n

First we shall encode the effect of hitting a specific whole. As an example lets use an 3 x 3 board.
We shall give each hole a unique number ie.

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \tag{1}$$

Now lets consider the effect of hitting hole 1, the holes 1, 2 and 4 would change state and all other holes would remain unchanged. We can represent this as a 1 x n^2 row vector.

$$\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{matrix}$$
$$\begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$
hole 1

Alternatively if we were to consider hitting hole 5, holes 2, 4, 6, 8 would change state and all other holes would remain unchanged.

$$\begin{matrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{matrix}$$
$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$
hole 5

If we stack these row vectors for all holes we arrive at a full n^2 x n^2 effects matrix

$$A = \begin{array}{c|ccccccccc} & 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \hline 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 2 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 3 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 4 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 5 & 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 \\ 6 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \\ 7 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 8 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \\ 9 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 \end{array}$$

Now lets define a new n^2 x 1 vector S, that defines the initial condition. e.g

$$\begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \end{bmatrix} \longrightarrow S = \begin{array}{ccccccccc} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \\ \multicolumn{9}{c}{\begin{bmatrix} 1 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 1 \end{bmatrix}^{\mathsf{T}}} \end{array}$$

Therefore we are looking for an x (a sequence of holes we can hit), that satifies $Ax = S$
Each time we hit a hole, that hole and the adjacent ones change states between states: a visible mole (1) becomes hidden (0) and vice versa. If we hit the same hole again all affected holes change back and the board returns to exactly the same state it was in before.

This means that hitting a hole an even number of times has no overall affect, while hitting it an odd number of times changes its state once. Therefore we can descirbe this using the XOR operation, which represents the hit once to change, hit twice to cancel rule:

$$1 \text{ XOR } 1 = 0, \quad 1 \text{ XOR } 0 = 1, \quad 0 \text{ XOR } 1 = 1, \quad 0 \text{ XOR } 0 = 0.$$

This can we solved using Gaussian elimination using XOR row operations.

$$A|S = \left[\begin{array}{ccccccccc|c} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 \end{array}\right]$$

An example would be, $R_4 \leftarrow \mathrm{R}_1 \oplus R_4$ (using XOR operation)

$$R_1 = \left[\begin{array}{ccccccccc|c} 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right],$$

$$R_4 = \left[\begin{array}{ccccccccc|c} 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{array}\right],$$

$$R_1 \oplus R_4 = \left[\begin{array}{ccccccccc|c} 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 0 \end{array}\right] = R_4^{(\mathrm{new})}$$

If we repeat this process until we obtain the identity matrix, I, on the left hand side we have successfully solved the problem, and we can read off vector x from the right. Alternatively if we end up with a contradiction there is no solution.

Contradiction: $\left[\begin{array}{ccccccccc|c} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{array}\right] \Rightarrow$

The system is inconsistent and there is no solution.

This fully describes our algorithm. In terms of complexity, when we compute our matrix A, we are computing an n^2 x n^2 matrix therefore we have $O(n^4)$. We then do guassian elemination which take $O(m^3)$ for an m x m matrix, therfore the guassian eleminatoin takes $O((n^2)^3) = O(n^6)$.

Our entire algorithm therefore takes $O(n^4) + O(n^6) = O(n^6)$ which is polynomial in n